DATABASE PROJECT · DESIGN, GENERATION & VALIDATION

Synthetic diabetes care database

A reproducible SQLite cohort for relational SQL practice, data-quality auditing, and responsible synthetic-data design.

Student ID	25023355
Git repository URL	https://github.com/Kundetisowmya/data_mining

1,200	6,057	13,216	11
synthetic patients	appointments	measurement rows	related tables

Project overview

This project models a fictional primary-care cohort of adults living with type 2 diabetes. Related tables represent clinics, clinicians, patients, conditions, appointments, observations, medications, and prescriptions. The fixed seed makes the database reproducible; deliberately seeded missing and duplicated values make data-quality work visible and measurable.

Data inventory

Table	Rows	Purpose
appointment	6,057	Care events and outcomes
clinic	6	Fictional care locations
clinician	30	Synthetic staff records
clinician_site	30	Staff-location assignments
condition_catalog	6	Condition lookup
dataset_metadata	7	Seed, cut-off, and policy notes
medication_catalog	8	Product/strength lookup
observation	13,216	Dated numeric measurements
patient	1,200	Cohort and demographics
patient_condition	3,104	Longitudinal diagnoses
prescription	1,921	Medication history

Data cut-off: 31 October 2026. All people, clinics, events, contact details, and clinical measurements are synthetic.

What this schema supports

QUERY	AUDIT	REPRODUCE
Join the cohort to diagnoses, appointments, measurements, and prescription history.	Find deliberate nulls and repeated contact or observation records with SQL.	Regenerate the same synthetic dataset from the fixed seed and verify its constraints.

01 / Schema & generation

Relational structure with purpose

The schema separates reference data from events and history. Patient and clinic identifiers link encounters to a registered site; clinician assignments ensure each encounter references a provider linked to that site. Measurements point to a composite (appointment, patient) key, preventing a measurement from being attached to an unrelated patient's visit.

Keys and constraints

Table	Key / relationship	Reason
patient_condition	PK: patient + condition + diagnosis date	Allows condition history while blocking an identical repeat row.
clinician_site	PK: clinician + clinic + start date	Records a clinician's clinic assignment; unique clinician/clinic pair supports encounter FK.
medication_catalog	PK: medication code + strength	Treats strengths as distinct reference items.
prescription	PK: patient + medication + strength + date	Allows one patient to have successive prescriptions over time.
observation	Surrogate PK + composite appointment/patient FK	Preserves duplicate source-like measures while enforcing a valid visit relationship.

Additional `CHECK` constraints restrict demographic categories, appointment outcomes, medication status, measurement units, and broad numeric ranges. `NOT NULL` is used for identifiers and required event fields; selected demographics and contact details remain nullable so missingness can be represented. Indexes support common patient/date, clinic/status, and patient/measure/date queries.

Generation approach

`generate_database.py` uses Python's standard library, a local seeded pseudorandom generator (`20261007`), code-defined fictional names and clinics, and a fixed data cut-off. Patient ages, comorbid conditions, appointment counts and outcomes, medication histories, and measurement values are drawn from bounded distributions. A small number of repeated contact emails and cloned measurement rows are inserted explicitly after base records are generated. Re-running the same seed reproduces the same data and the same quality-audit counts.

Measurement values use broad simulated ranges to create plausible database examples; they are not validated against a clinical reference population and must not be interpreted as targets or care guidance.

Database structure

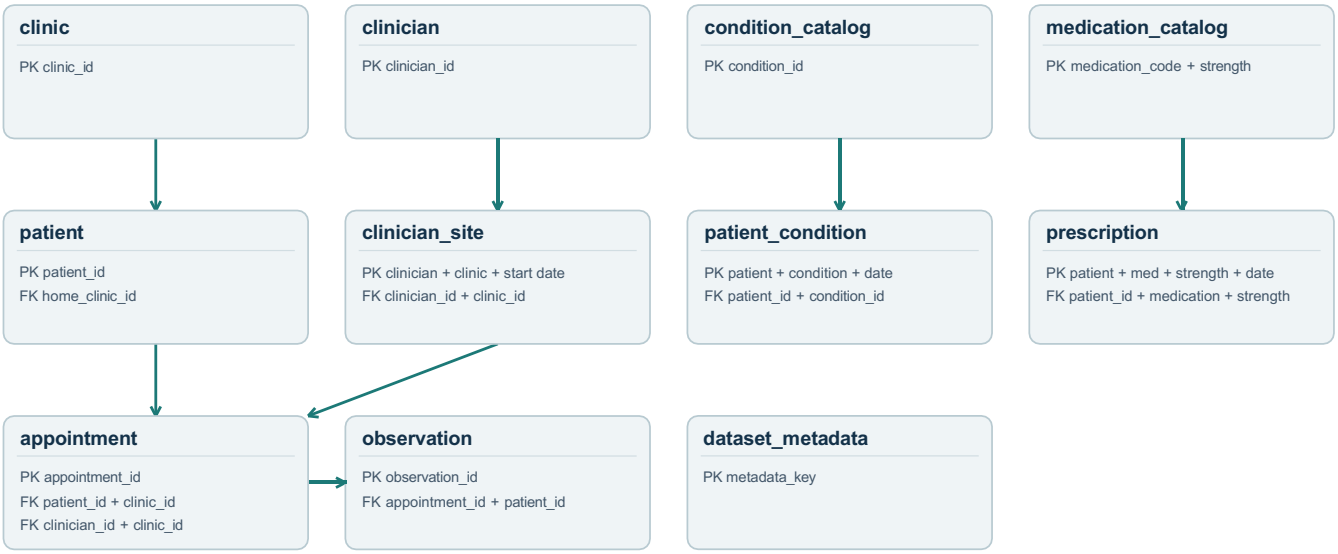


Figure 1. Core tables and key fields. PK = primary key; FK = foreign key. Arrows show major parent-to-child paths; composite links are named in each table card.

02 / Quality, validation & privacy

Visible defects, verified relationships

The dataset intentionally keeps selected field-level omissions and repeated values. They are documented here and asserted by the generator, so they are reproducible assessment features rather than accidental side effects.

Seeded data-quality issues

Issue	Count	Rate / interpretation
Missing contact email	36	3.0% of patients; null contact field.
Missing smoking status	60	5.0% of patients; not recorded.
Missing ethnicity	36	3.0% of patients; not recorded.
Repeated email values	12 values / 24 patients	2.0% of patients share a contact address in 12 pairs.
Cloned observations	36 extra rows	36 duplicate event groups; 0.27% of measurement rows.

The repeated observation business key is appointment, patient, measure, date, numeric value, and unit. Rows have different observation IDs. No uniqueness constraint is applied to that business key so a learner can detect and handle the duplicate explicitly.

Validation results

SQLite `integrity_check`: **ok**. `foreign_key_check`: **0 violations**. At least one table contains more than 1,000 rows. The generator confirmed every seeded missing and duplicate count, then ran six savepoint-based invalid-insert probes. Category/range `CHECK`, `UNIQUE`, `NOT NULL`, foreign-key, and composite-primary-key rules all rejected their probes; the probes were rolled back.

Privacy and responsible use

No real patient data, records, or source extracts were used. Names and organizations are fictional. Email-like values use the reserved `.invalid` suffix and cannot be used as real contact details. Synthetic IDs carry no external identity mapping. Demographic categories and distributions are simplified and should not be treated as representative. The database is for coursework and SQL practice only, not for diagnosis, treatment, service planning, or conclusions about real communities.

Reproduce and submit

Run `python3 generate_database.py --db-path synthetic_diabetes.db --seed 20261007`; the script prints a machine-readable validation summary. Open and run `SQL_Project_Assessment.ipynb` from the same folder to see design explanations, audit queries, and relational SQL examples.

Package contents

File	Purpose
synthetic_diabetes.db	Ready-to-query SQLite database.
generate_database.py	Deterministic generator and validation output.
SQL_Project_Assessment.ipynb	Design notes, audit queries, and SQL examples.
SQL_Project_Report.pdf	Schema, quality, privacy, and validation summary.
README.md	Reproduction and submission instructions.

Personal review required: add your student ID and Git repository URL in the spaces on page 1, compare the submission with the full course rubric, include any required AI-use declaration, and submit the ZIP through the course portal yourself.